

MEDITrack

Hospital Appointment & Doctor Scheduling System

Database Management System (DBMS) Project Report

Submitted by	Omkar Narvekar
Roll Number	150096752147
Course	B.Tech Computer Science Engineering (2025-29), Semester III
Subject	Database Management Systems (SQL & NoSQL)
Case Study	91. MediTrack (Hospital Appointment & Doctor Scheduling System)
Technology	PostgreSQL and pgAdmin 4
Academic Year	2026-2027

Abstract

MediTrack is a Hospital Appointment and Doctor Scheduling System developed using PostgreSQL. Its objective is to manage hospital departments, doctors, patients, appointments and prescriptions in a relational database.

The system stores information in five related tables: Department, Doctor, Patient, Appointment and Prescription. Primary keys and foreign keys maintain the relationships between them and preserve data integrity.

The project demonstrates SQL operations such as CREATE, INSERT, SELECT, JOIN, GROUP BY, aggregate functions and subqueries. It also uses an index to support appointment-date searches and a trigger that prevents overlapping appointments for the same doctor.

1. Introduction

Hospitals manage large amounts of information about patients, doctors, departments, appointments and prescriptions. Managing this information manually can lead to duplication, errors and difficulty retrieving records. MediTrack provides a database-based solution for storing and retrieving this information. PostgreSQL is the database management system, and pgAdmin 4 provides the graphical interface for executing SQL and managing database objects.

2. Problem Statement

MediTrack requires a database for a hospital that allows patients to book appointments with doctors based on available time slots, while preventing two patients from booking the same doctor at overlapping times. Doctor department details and department contact information must not be repeated for every doctor, so Department is separated into its own table to remove redundancy and achieve 3NF.

3. Objectives

- Store patient and doctor information.
- Maintain hospital department records.
- Manage appointment dates, timings and statuses.
- Store prescription information.
- Retrieve related information using JOIN queries and display available doctors for a requested date and time.
- Generate reports using aggregate functions and subqueries.
- Prevent overlapping appointments for the same doctor using a trigger.

4. Software and Hardware Requirements

Component	Description
Operating System	macOS
Database	PostgreSQL
Database Interface	pgAdmin 4
Query Language	SQL

Component	Description
Procedural Language	PL/pgSQL
Report Preparation	PDF

- Laptop or desktop computer; minimum 4 GB RAM recommended.
- Sufficient storage for PostgreSQL and project files.

5. Scope of the Project

The project covers the database layer of a hospital appointment system: data storage, relationships, appointment scheduling, prescription records and SQL-based reporting. The implementation uses sample data and does not include a patient-facing website, online payments, authentication or live hospital integration.

6. Database Design

Database name: **meditrack**. It contains five tables.

6.1 Department

Column	Type	Description
DepartmentID	SERIAL	Primary key
DepartmentName	VARCHAR(100)	Unique, not null
ContactNumber	VARCHAR(15)	Department contact number
Location	VARCHAR(100)	Department location

6.2 Doctor

Column	Type	Description
DoctorID	SERIAL	Primary key
DoctorName	VARCHAR(100)	Doctor's name
Specialization	VARCHAR(100)	Medical specialization
PhoneNumber	VARCHAR(15)	Unique
Email	VARCHAR(100)	Unique
DepartmentID	INTEGER	Foreign key to Department

6.3 Patient

Column	Type	Description
PatientID	SERIAL	Primary key
PatientName	VARCHAR(100)	Patient's name
Age	INTEGER	Patient's age (CHECK)
Gender	VARCHAR(20)	Patient's gender
PhoneNumber	VARCHAR(15)	Unique
Email	VARCHAR(100)	Unique
Address	TEXT	Patient's address

6.4 Appointment

Column	Type	Description
AppointmentID	SERIAL	Primary key
PatientID	INTEGER	Foreign key to Patient
DoctorID	INTEGER	Foreign key to Doctor
AppointmentDate	DATE	Appointment date
StartTime	TIME	Start time
EndTime	TIME	End time (CHECK: after StartTime)
Status	VARCHAR(20)	Scheduled, Completed or Cancelled (CHECK)
Reason	TEXT	Reason for the appointment

6.5 Prescription

Column	Type	Description
PrescriptionID	SERIAL	Primary key
AppointmentID	INTEGER	Foreign key to Appointment
MedicationName	VARCHAR(100)	Medication name
Dosage	VARCHAR(100)	Prescribed dosage
Frequency	VARCHAR(100)	Frequency of use
Duration	VARCHAR(100)	Duration of treatment
Notes	TEXT	Additional notes
PrescriptionDate	DATE	Date of prescription

7. Entity Relationship Diagram

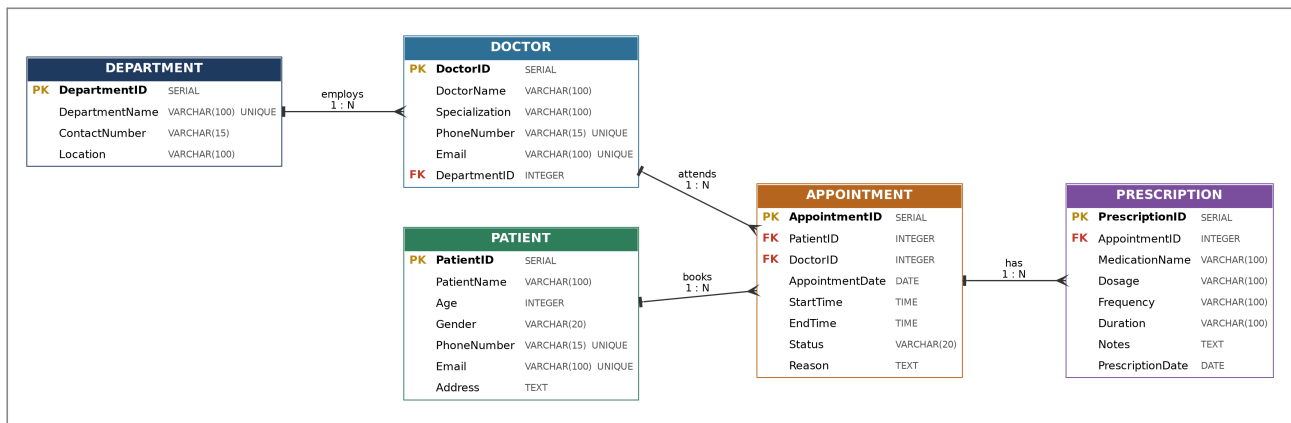


Figure 1 - Entity Relationship Diagram of MediTrack (PK = primary key, FK = foreign key; crow's foot = many side, bar = one side).

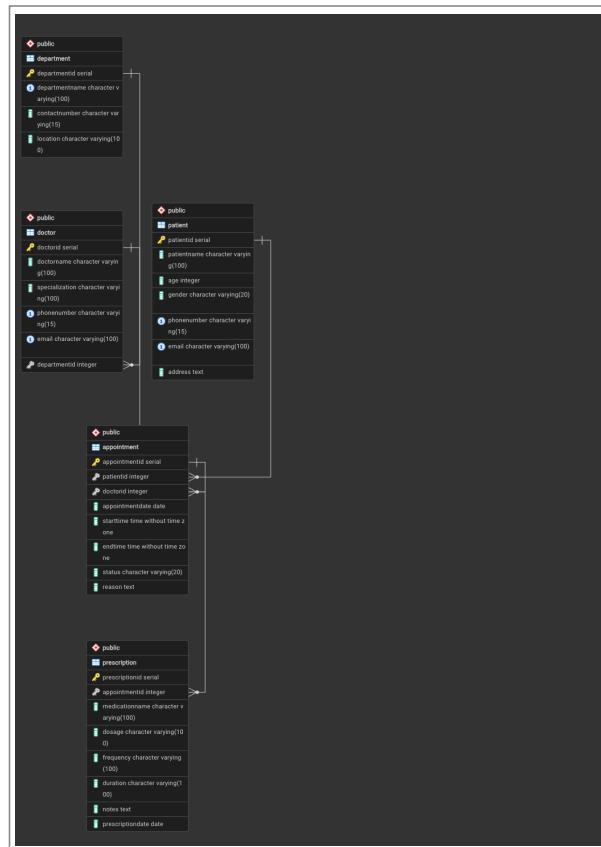


Figure 2 - Schema diagram generated by pgAdmin 4 for the same database.

Relationships

- Department to Doctor: one department has many doctors (Doctor.DepartmentID references Department).
- Doctor to Appointment: one doctor has many appointments (Appointment.DoctorID references Doctor).

- Patient to Appointment: one patient can book many appointments (Appointment.PatientID references Patient).
- Appointment to Prescription: one appointment can have many prescription records (Prescription.AppointmentID references Appointment).

These foreign keys ensure that every appointment belongs to a valid patient and doctor (referential integrity) and avoid repeating department, doctor and patient details in appointment records.

8. Normalization

First Normal Form (1NF). Every column holds a single atomic value; there are no repeating groups or lists in one field, and each row is identified by a primary key.

Second Normal Form (2NF). All tables use single-column primary keys, so no non-key attribute can depend on part of a composite key.

Third Normal Form (3NF). Department name, contact number and location are stored once in the Department table rather than repeated with every doctor. Doctor, patient, appointment and prescription details are separated into their own tables, so non-key attributes depend only on the key of their own table.

Before and after. A single un-normalized table would repeat the department name and contact number on every doctor row (and every doctor detail on every appointment row). Splitting Department out removes that redundancy and the update anomalies it causes.

9. SQL Implementation

9.1 Data Definition Language (DDL)

DDL defines the structure. Example tables:

```
CREATE TABLE Department (
    DepartmentID SERIAL PRIMARY KEY,
    DepartmentName VARCHAR(100) UNIQUE NOT NULL,
    ContactNumber VARCHAR(15),
    Location VARCHAR(100)
);

CREATE TABLE Appointment (
    AppointmentID SERIAL PRIMARY KEY,
    PatientID INTEGER NOT NULL REFERENCES Patient(PatientID),
    DoctorID INTEGER NOT NULL REFERENCES Doctor(DoctorID),
    AppointmentDate DATE NOT NULL,
    StartTime TIME NOT NULL,
    EndTime TIME NOT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Scheduled'
    CHECK (Status IN ('Scheduled', 'Completed', 'Cancelled')),
    Reason TEXT,
    CHECK (EndTime > StartTime)
);
```

9.2 Data Manipulation Language (DML)

INSERT INTO is used to add sample records: five departments, five doctors, five patients, five appointments and five prescriptions.

9.3 Constraints

- **PRIMARY KEY** uniquely identifies each record.

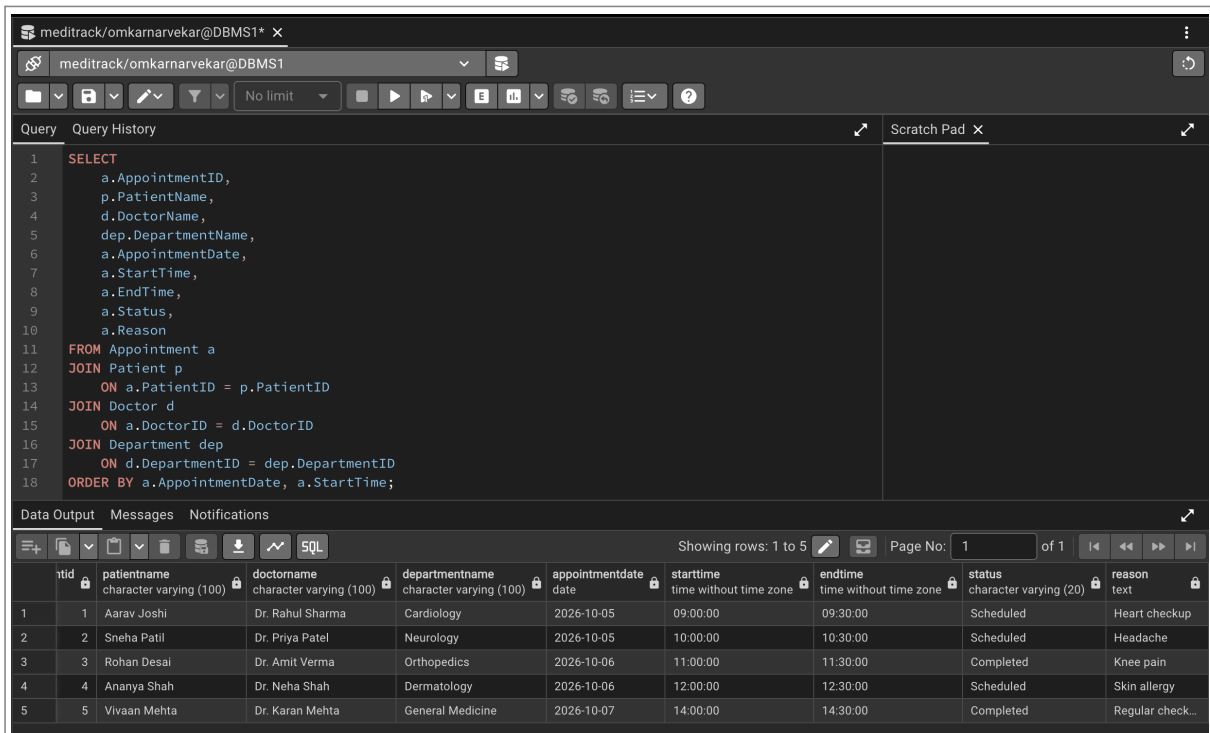
- **FOREIGN KEY** connects related tables and enforces referential integrity.
- **NOT NULL** requires a value.
- **UNIQUE** prevents duplicate department names, phone numbers and emails.
- **CHECK** validates age, appointment status and that EndTime is later than StartTime.
- **DEFAULT** supplies default values (for example Status = 'Scheduled').

10. SQL Queries and Reports

10.1 JOIN query: appointment details

Displays each appointment together with patient, doctor and department details.

```
SELECT a.AppointmentID, p.PatientName, d.DoctorName, dep.DepartmentName,
       a.AppointmentDate, a.StartTime, a.EndTime, a.Status, a.Reason
FROM Appointment a
JOIN Patient p      ON a.PatientID = p.PatientID
JOIN Doctor d       ON a.DoctorID = d.DoctorID
JOIN Department dep ON d.DepartmentID = dep.DepartmentID
ORDER BY a.AppointmentDate, a.StartTime;
```



The screenshot shows a database query tool interface. The top section displays the SQL query for appointment details. The bottom section shows the query results in a table format. The table has 10 columns: AppointmentID, PatientName, DoctorName, DepartmentName, AppointmentDate, StartTime, EndTime, Status, Reason, and a lock icon. The results show 5 rows of appointment data.

AppointmentID	PatientName	DoctorName	DepartmentName	AppointmentDate	StartTime	EndTime	Status	Reason
1	Aarav Joshi	Dr. Rahul Sharma	Cardiology	2026-10-05	09:00:00	09:30:00	Scheduled	Heart checkup
2	Sneha Patil	Dr. Priya Patel	Neurology	2026-10-05	10:00:00	10:30:00	Scheduled	Headache
3	Rohan Desai	Dr. Amit Verma	Orthopedics	2026-10-06	11:00:00	11:30:00	Completed	Knee pain
4	Ananya Shah	Dr. Neha Shah	Dermatology	2026-10-06	12:00:00	12:30:00	Scheduled	Skin allergy
5	Vivaan Mehta	Dr. Karan Mehta	General Medicine	2026-10-07	14:00:00	14:30:00	Completed	Regular check...

Figure 3 - Output of the appointment JOIN query (5 rows).

10.2 JOIN query: prescriptions

```
SELECT p.PatientName, d.DoctorName, a.AppointmentDate,
       pr.MedicationName, pr.Dosage, pr.Frequency, pr.Duration, pr.Notes
FROM Prescription pr
JOIN Appointment a ON pr.AppointmentID = a.AppointmentID
JOIN Patient p     ON a.PatientID = p.PatientID
JOIN Doctor d      ON a.DoctorID = d.DoctorID
ORDER BY a.AppointmentDate;
```

The screenshot shows a database query editor with the following SQL query:

```

1  SELECT
2      p.PatientName,
3      d.DoctorName,
4      a.AppointmentDate,
5      pr.MedicationName,
6      pr.Dosage,
7      pr.Frequency,
8      pr.Duration,
9      pr.Notes
10 FROM Prescription pr
11 JOIN Appointment a
12     ON pr.AppointmentID = a.AppointmentID
13 JOIN Patient p
14     ON a.PatientID = p.PatientID
15 JOIN Doctor d
16     ON a.DoctorID = d.DoctorID
17 ORDER BY a.AppointmentDate;

```

The results table displays 5 rows of data:

	patientname character varying (100)	doctormame character varying (100)	appointmentdate date	medicationname character varying (100)	dosage character varying (100)	frequency character varying (100)	duration character varying (100)	notes text
1	Aarav Joshi	Dr. Rahul Sharma	2026-10-05	Aspirin	75 mg	Once daily	5 days	Take after food
2	Sneha Patil	Dr. Priya Patel	2026-10-05	Paracetamol	500 mg	Twice daily	3 days	For headache
3	Rohan Desai	Dr. Amit Verma	2026-10-06	Ibuprofen	200 mg	Twice daily	5 days	Take after food
4	Ananya Shah	Dr. Neha Shah	2026-10-06	Cetirizine	10 mg	Once daily	5 days	For allergy sympto...
5	Vivaan Mehta	Dr. Karan Mehta	2026-10-07	Vitamin D3	1000 IU	Once daily	30 days	Take with a meal

Figure 4 - Prescriptions with patient and doctor details (5 rows).

10.3 Aggregate query: doctor-wise appointment count

```

SELECT d.DoctorName, d.Specialization,
       COUNT(a.AppointmentID) AS TotalAppointments
FROM Doctor d
LEFT JOIN Appointment a ON d.DoctorID = a.DoctorID
GROUP BY d.DoctorID, d.DoctorName, d.Specialization
ORDER BY TotalAppointments DESC;

```

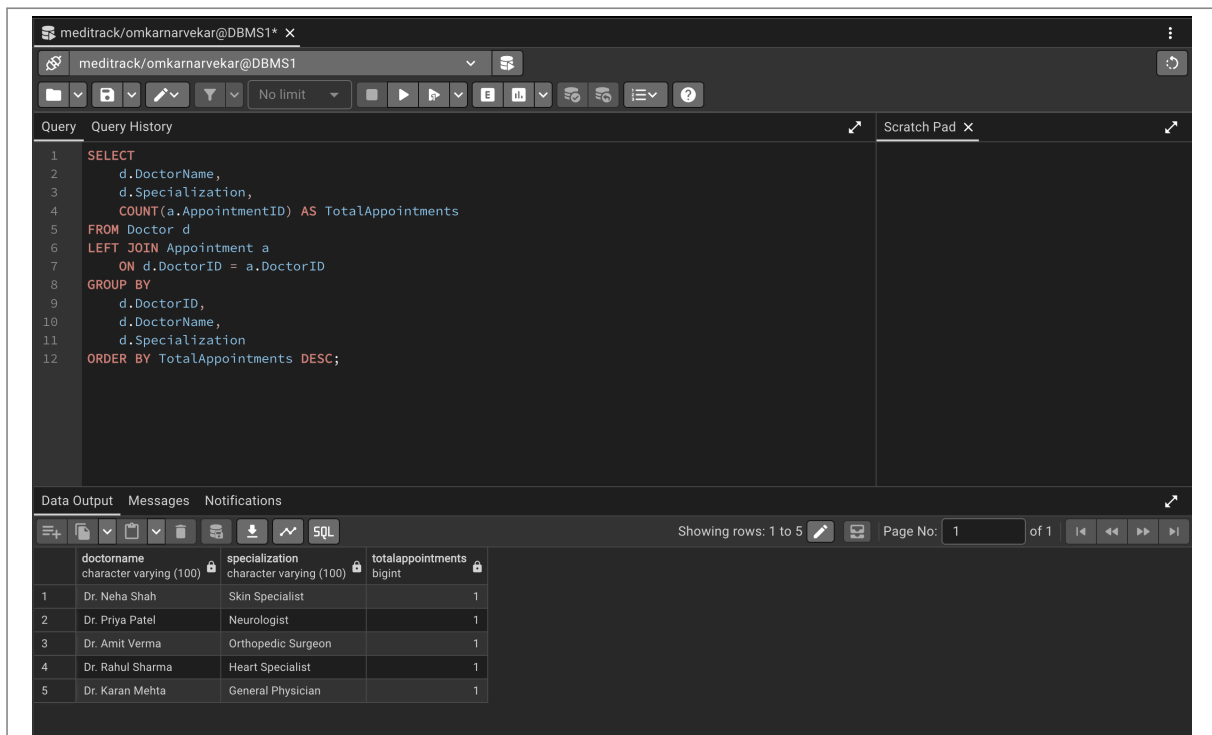


Figure 5 - Appointment count per doctor (each doctor has 1 appointment in the sample data).

10.4 Aggregate query: department-wise appointment count

```

SELECT dep.DepartmentName, COUNT(a.AppointmentID) AS TotalAppointments
FROM Department dep
LEFT JOIN Doctor d      ON dep.DepartmentID = d.DepartmentID
LEFT JOIN Appointment a ON d.DoctorID = a.DoctorID
GROUP BY dep.DepartmentID, dep.DepartmentName
ORDER BY TotalAppointments DESC;

```

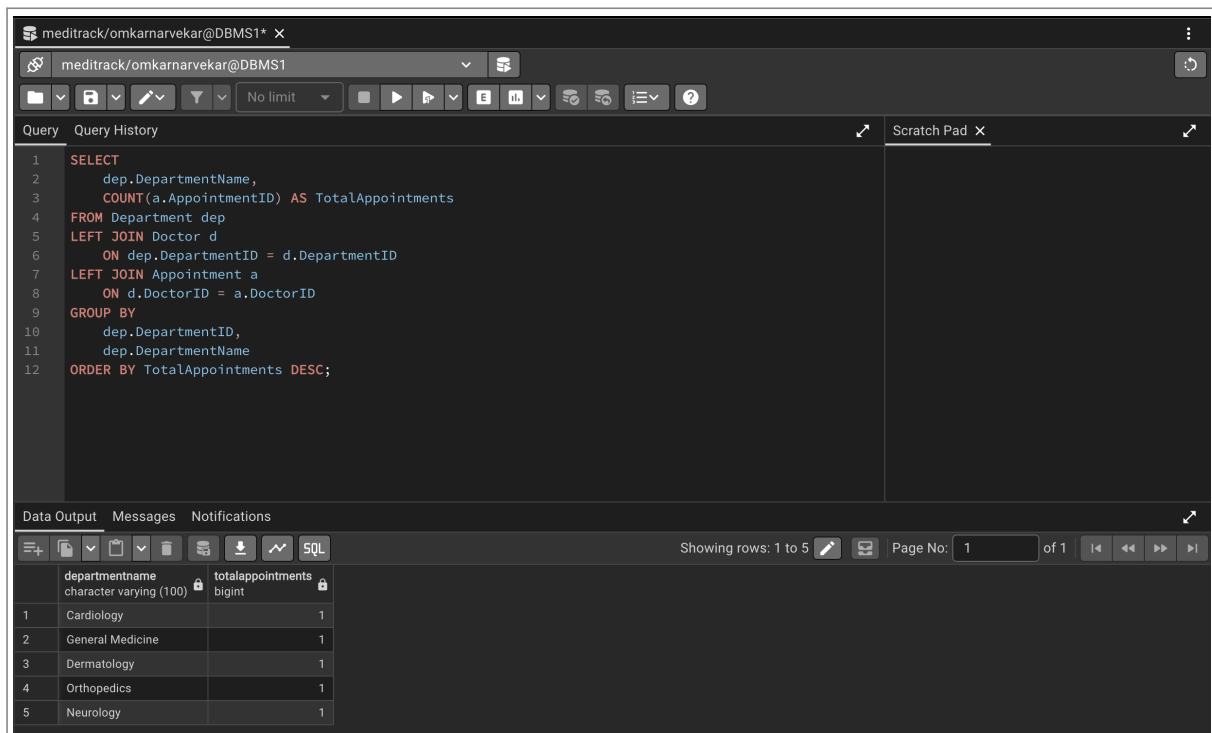


Figure 6 - Appointment count per department.

10.5 Subquery: patients who have appointments

```

SELECT PatientID, PatientName, PhoneNumber, Email
FROM Patient
WHERE PatientID IN (SELECT PatientID FROM Appointment);

```

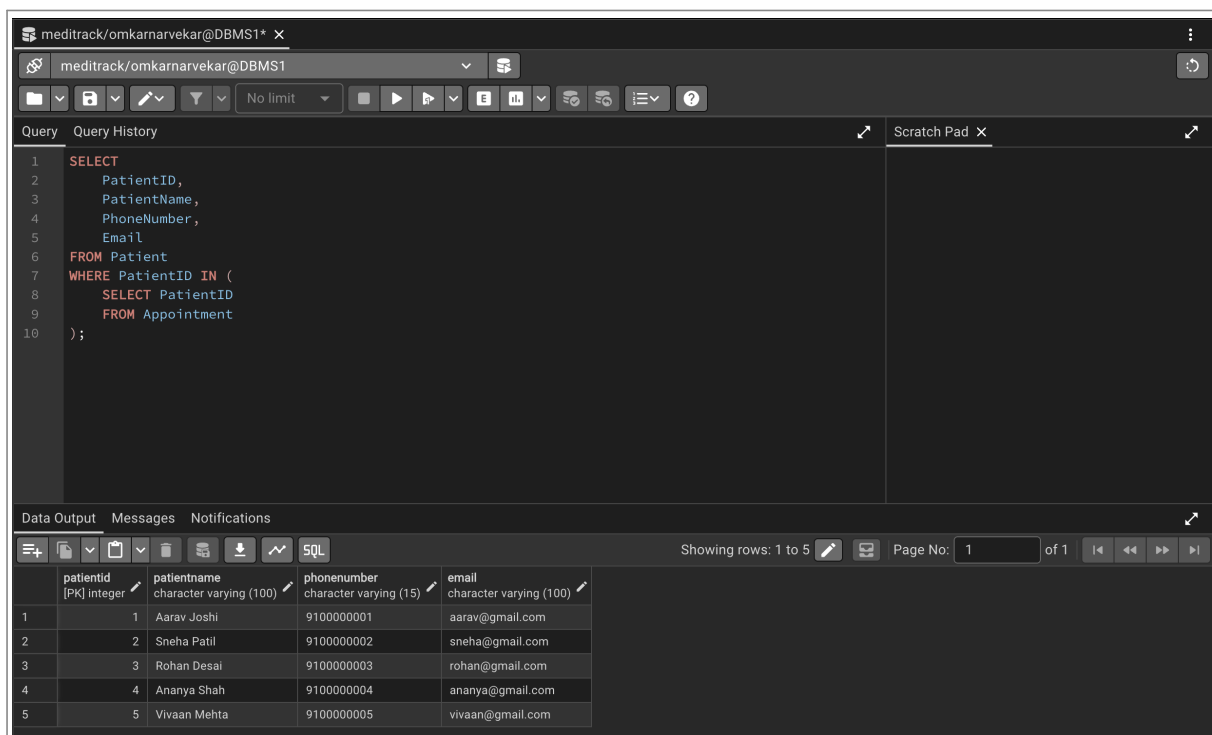
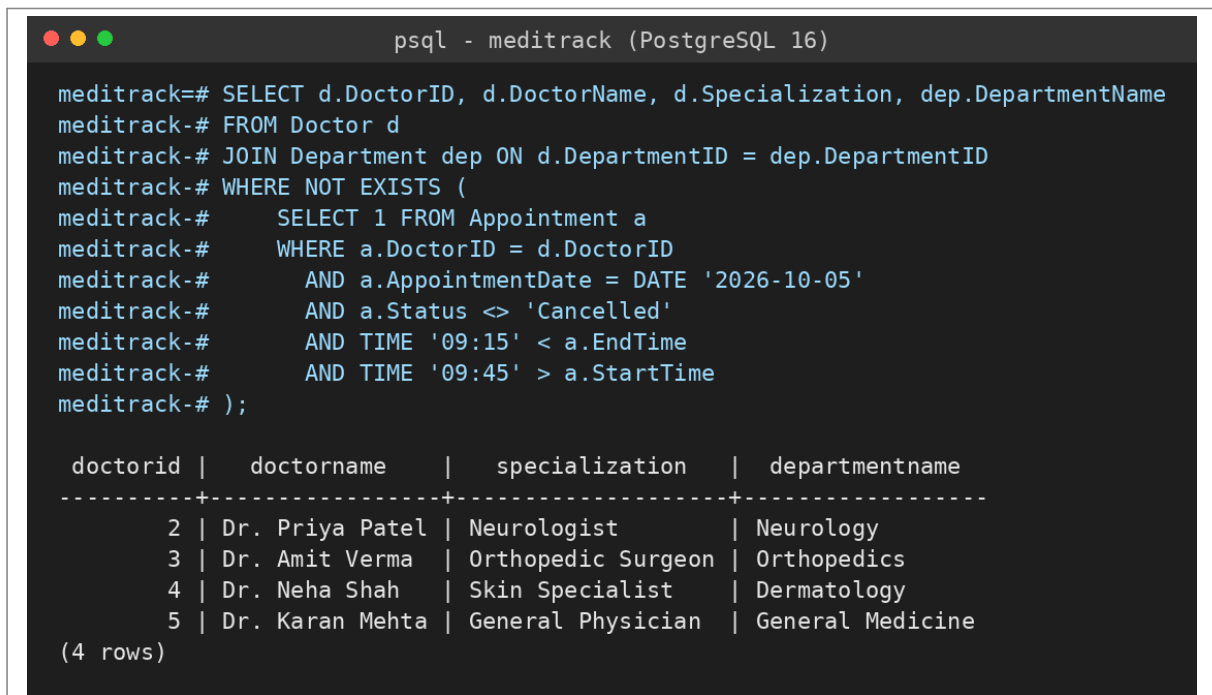


Figure 7 - Patients with at least one appointment (5 rows).

10.6 Available doctors for a requested date and time

A doctor is available if he or she has no non-cancelled appointment that overlaps the requested slot. The example checks 2026-10-05 from 09:15 to 09:45.

```
SELECT d.DoctorID, d.DoctorName, d.Specialization, dep.DepartmentName
FROM Doctor d
JOIN Department dep ON d.DepartmentID = dep.DepartmentID
WHERE NOT EXISTS (
    SELECT 1
    FROM Appointment a
    WHERE a.DoctorID = d.DoctorID
    AND a.AppointmentDate = DATE '2026-10-05'
    AND a.Status <> 'Cancelled'
    AND TIME '09:15' < a.EndTime      -- requested start before existing end
    AND TIME '09:45' > a.StartTime   -- requested end after existing start
);
```



```
psql - meditrack (PostgreSQL 16)

meditrack=# SELECT d.DoctorID, d.DoctorName, d.Specialization, dep.DepartmentName
meditrack=# FROM Doctor d
meditrack=# JOIN Department dep ON d.DepartmentID = dep.DepartmentID
meditrack=# WHERE NOT EXISTS (
meditrack=#     SELECT 1 FROM Appointment a
meditrack=#     WHERE a.DoctorID = d.DoctorID
meditrack=#           AND a.AppointmentDate = DATE '2026-10-05'
meditrack=#           AND a.Status <> 'Cancelled'
meditrack=#           AND TIME '09:15' < a.EndTime
meditrack=#           AND TIME '09:45' > a.StartTime
meditrack=# );
```

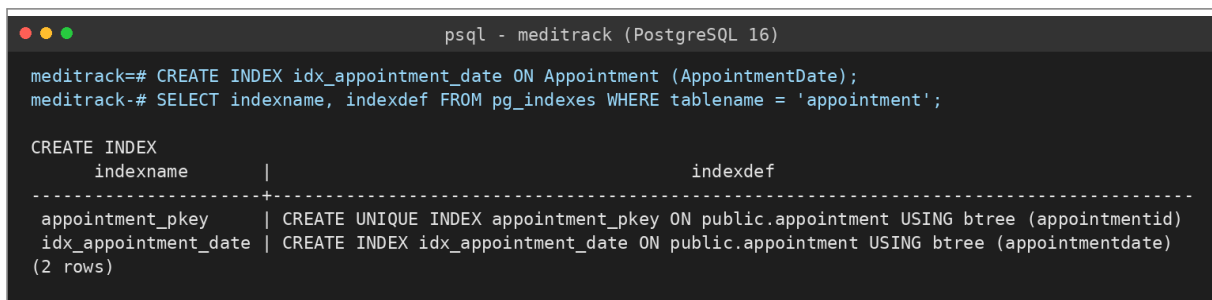
doctorid	doctorname	specialization	departmentname
2	Dr. Priya Patel	Neurologist	Neurology
3	Dr. Amit Verma	Orthopedic Surgeon	Orthopedics
4	Dr. Neha Shah	Skin Specialist	Dermatology
5	Dr. Karan Mehta	General Physician	General Medicine

(4 rows)

Figure 8 - Doctors available on 2026-10-05 from 09:15 to 09:45 (Dr. Rahul Sharma is excluded because of his 09:00-09:30 appointment).

10.7 Index

```
CREATE INDEX idx_appointment_date
ON Appointment (AppointmentDate);
```



```
psql - meditrack (PostgreSQL 16)

meditrack=# CREATE INDEX idx_appointment_date ON Appointment (AppointmentDate);
meditrack=# SELECT indexname, indexdef FROM pg_indexes WHERE tablename = 'appointment';
```

indexname	indexdef
appointment_pkey	CREATE UNIQUE INDEX appointment_pkey ON public.appointment USING btree (appointmentid)
idx_appointment_date	CREATE INDEX idx_appointment_date ON public.appointment USING btree (appointmentdate)

(2 rows)

Figure 9 - Creating idx_appointment_date and listing the indexes on the Appointment table.

11. Trigger: Preventing Double Booking

A BEFORE INSERT OR UPDATE trigger rejects an appointment whose time overlaps another appointment for the same doctor on the same date. Two time ranges overlap when the new start is before the existing end and the new end is after the existing start. Cancelled appointments are ignored, both for the new row and for existing rows, so a cancelled slot can be rebooked.

```
CREATE OR REPLACE FUNCTION prevent_overlapping_appointments()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.Status <> 'Cancelled' AND EXISTS (
        SELECT 1
        FROM Appointment a
        WHERE a.DoctorID = NEW.DoctorID
              AND a.AppointmentDate = NEW.AppointmentDate
              AND a.Status <> 'Cancelled'
              AND a.AppointmentID IS DISTINCT FROM NEW.AppointmentID
              AND NEW.StartTime < a.EndTime
              AND NEW.EndTime > a.StartTime
    ) THEN
        RAISE EXCEPTION
        'Doctor % already has an appointment overlapping % - % on %',
        NEW.DoctorID, NEW.StartTime, NEW.EndTime, NEW.AppointmentDate;
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_overlap
BEFORE INSERT OR UPDATE ON Appointment
FOR EACH ROW
EXECUTE FUNCTION prevent_overlapping_appointments();
```

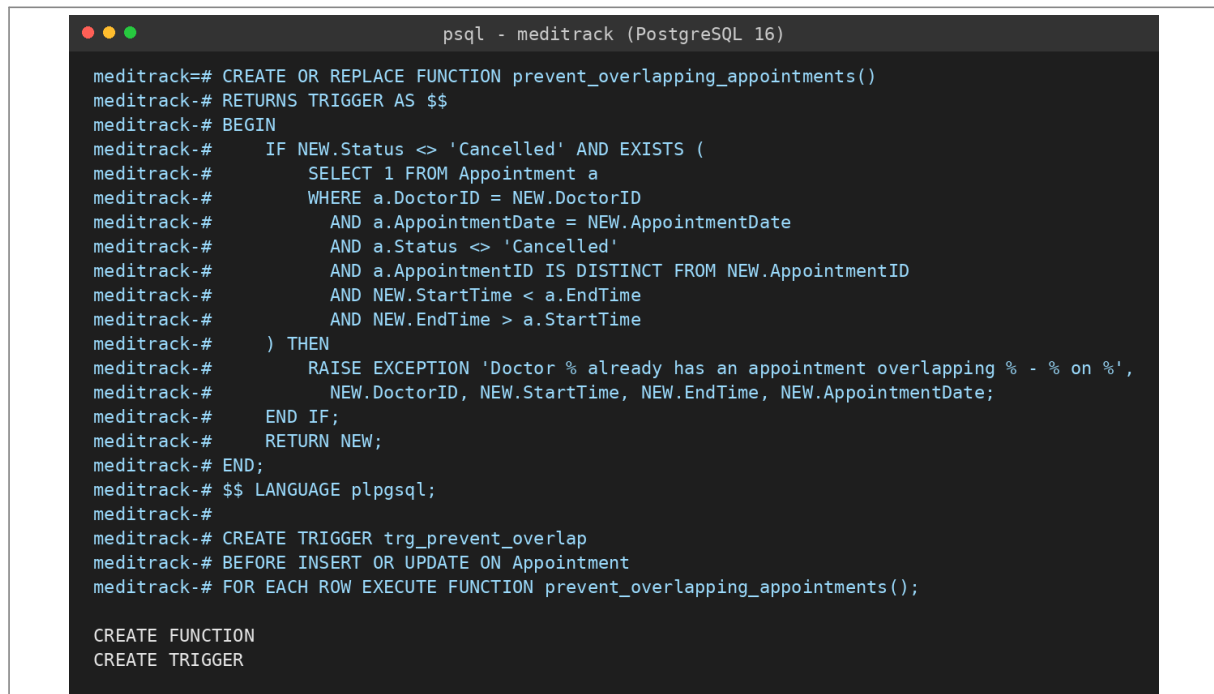
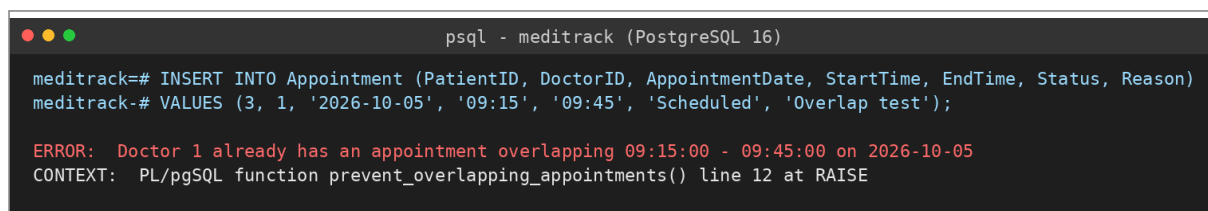
A screenshot of a PostgreSQL terminal window titled 'psql - meditrack (PostgreSQL 16)'. The terminal shows the execution of SQL commands to create a trigger function and a trigger. The commands are: 1. CREATE OR REPLACE FUNCTION prevent_overlapping_appointments() RETURNS TRIGGER AS \$\$ BEGIN IF NEW.Status <> 'Cancelled' AND EXISTS (SELECT 1 FROM Appointment a WHERE a.DoctorID = NEW.DoctorID AND a.AppointmentDate = NEW.AppointmentDate AND a.Status <> 'Cancelled' AND a.AppointmentID IS DISTINCT FROM NEW.AppointmentID AND NEW.StartTime < a.EndTime AND NEW.EndTime > a.StartTime) THEN RAISE EXCEPTION 'Doctor % already has an appointment overlapping % - % on %', NEW.DoctorID, NEW.StartTime, NEW.EndTime, NEW.AppointmentDate; END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql; 2. CREATE TRIGGER trg_prevent_overlap BEFORE INSERT OR UPDATE ON Appointment FOR EACH ROW EXECUTE FUNCTION prevent_overlapping_appointments(); The terminal output shows the commands being entered and the function being created.

Figure 10 - Creating the trigger function and the trigger.

Test 1 - overlapping insert (must be rejected)

Dr. Rahul Sharma (DoctorID 1) already has 09:00-09:30 on 2026-10-05, so this insert must fail:

```
INSERT INTO Appointment (PatientID, DoctorID, AppointmentDate, StartTime, EndTime, Status, Reason)
VALUES (3, 1, '2026-10-05', '09:15', '09:45', 'Scheduled', 'Overlap test');
```

A terminal window titled 'psql - meditrack (PostgreSQL 16)' showing the execution of an SQL insert statement. The statement is rejected by a trigger. The error message is: 'ERROR: Doctor 1 already has an appointment overlapping 09:15:00 - 09:45:00 on 2026-10-05 CONTEXT: PL/pgSQL function prevent_overlapping_appointments() line 12 at RAISE'.

```
psql - meditrack (PostgreSQL 16)
meditrack=# INSERT INTO Appointment (PatientID, DoctorID, AppointmentDate, StartTime, EndTime, Status, Reason)
meditrack=# VALUES (3, 1, '2026-10-05', '09:15', '09:45', 'Scheduled', 'Overlap test');

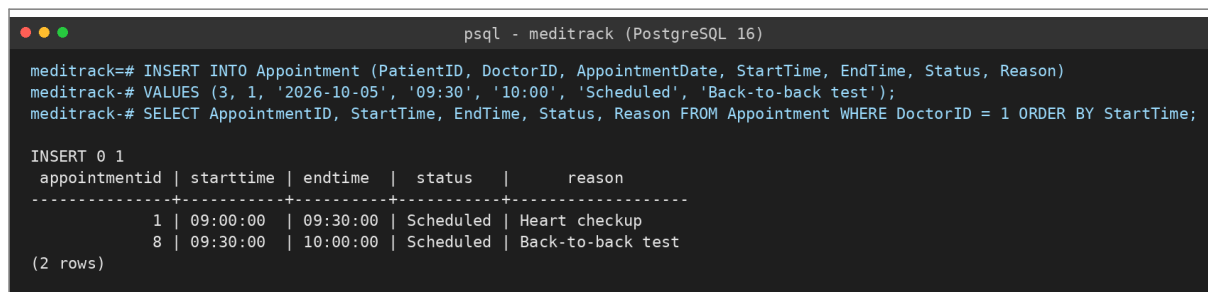
ERROR: Doctor 1 already has an appointment overlapping 09:15:00 - 09:45:00 on 2026-10-05
CONTEXT: PL/pgSQL function prevent_overlapping_appointments() line 12 at RAISE
```

Figure 11 - Overlapping insert (09:15-09:45) rejected by the trigger.

Test 2 - non-overlapping insert (must succeed)

```
INSERT INTO Appointment (PatientID, DoctorID, AppointmentDate, StartTime, EndTime, Status, Reason)
VALUES (3, 1, '2026-10-05', '09:30', '10:00', 'Scheduled', 'Back-to-back test');
```

```
SELECT AppointmentID, StartTime, EndTime, Status, Reason
FROM Appointment WHERE DoctorID = 1 ORDER BY StartTime;
```

A terminal window titled 'psql - meditrack (PostgreSQL 16)' showing the execution of an SQL insert statement followed by a query. The insert is successful. The query result shows two rows: one for a heart checkup and one for a back-to-back test.

```
psql - meditrack (PostgreSQL 16)
meditrack=# INSERT INTO Appointment (PatientID, DoctorID, AppointmentDate, StartTime, EndTime, Status, Reason)
meditrack=# VALUES (3, 1, '2026-10-05', '09:30', '10:00', 'Scheduled', 'Back-to-back test');
meditrack=# SELECT AppointmentID, StartTime, EndTime, Status, Reason FROM Appointment WHERE DoctorID = 1 ORDER BY StartTime;

INSERT 0 1
 appointmentid | starttime | endtime | status |      reason
-----+-----+-----+-----+-----
          1 | 09:00:00 | 09:30:00 | Scheduled | Heart checkup
          8 | 09:30:00 | 10:00:00 | Scheduled | Back-to-back test
(2 rows)
```

Figure 12 - Back-to-back insert (09:30-10:00) accepted. AppointmentID values may differ in your database.

12. Testing and Results

The database was tested with SQL queries on PostgreSQL. Figures 3-7 are pgAdmin 4 screenshots; Figures 8-12 (availability query, index, trigger and trigger tests) are psql console output from running the same scripts against the same schema and sample data.

Test	Expected result	Evidence
Create tables	Five tables created	Figures 1, 2 (ER diagrams)
Insert sample data	Records inserted	Figures 3-7
JOIN queries	Related records displayed	Figures 3, 4
Aggregate queries	Appointment counts displayed	Figures 5, 6
Subquery	Matching patients returned	Figure 7
Available doctors query	Doctors without overlapping appointments listed	Figure 8
Create index	Appointment-date index created	Figure 9
Overlap trigger	Conflicting appointment rejected	Figures 10, 11, 12
Foreign keys	Relationships displayed	Figures 1, 2

Outcomes against the case study. Doctor double-booking is prevented by the trigger; patients can find available doctors for a slot using the availability query; doctor-wise appointment counts are generated by the aggregate query; and foreign keys ensure every appointment belongs to a valid patient and doctor.

13. Conclusion

MediTrack demonstrates the use of PostgreSQL to build a relational database for hospital appointment and doctor scheduling. The project includes five related tables in 3NF, primary and foreign keys, constraints, sample records, JOIN, aggregate and subquery reports, an index and an overlap-prevention trigger written in PL/pgSQL. It provides a foundation for a more complete hospital management application.

14. Future Scope

- A web interface for patients and doctors.
- User authentication and role-based access.
- Doctor availability and working-hour management.
- Appointment booking and cancellation screens.
- Prescription printing.
- Billing and payment records.
- Automated reminders and analytics.

15. References

- PostgreSQL Official Documentation - <https://www.postgresql.org/docs/>
- pgAdmin Official Documentation - <https://www.pgadmin.org/docs/>

- PostgreSQL SQL Commands - <https://www.postgresql.org/docs/current/sql-commands.html>
- PostgreSQL PL/pgSQL - <https://www.postgresql.org/docs/current/plpgsql.html>